

A Recursive Hybrid Model for Piecewise Linear Regression: Algorithm Design, Analysis, and Empirical Evaluation

[Author(s) — Affiliation(s)]

Abstract

We present the Recursive Hybrid Model (RHM), a Linear Model Tree algorithm that recursively partitions the feature space using binary splits and fits a local Ridge regression model at each leaf. RHM belongs to the well-established family of Model Trees—algorithms that combine decision-tree partitioning with linear leaf models—a lineage traceable to Quinlan’s M5 (1992) and studied extensively since. Within this family, RHM’s distinguishing design choice is a Ridge-regression-guided feature selection step: rather than conducting an exhaustive search over all splitting features, RHM fits a Ridge regression at each internal node and selects the feature with the largest absolute coefficient as the splitting axis. This heuristic reduces the per-node feature selection cost from $O(nd^2)$ per feature to a single Ridge fit, while remaining data-driven. Split points are then optimised over a percentile grid using a weighted MSE criterion. We provide a precise algorithmic description, complexity analysis, and an empirical evaluation on synthetic data with linear, quadratic, and interaction structure. Results confirm that RHM consistently outperforms a global Ridge baseline, with accuracy increasing with tree depth before diminishing returns emerge.

1 Introduction

Many real-world regression problems exhibit regime-dependent behaviour: the relationship between predictors and response is approximately linear within local regions of the feature space but differs substantially across those regions. Globally linear models such as Ridge or Lasso regression cannot capture these local variations, while fully non-parametric methods such as Gaussian Processes may be computationally prohibitive at scale.

Decision-tree-based methods handle partitioning naturally, but classical regression trees (CART; Breiman et al. 1984) assign a constant value at each leaf—a poor approximation when the within-region signal is itself linear. The class of Linear Model Trees (also called Model Trees) addresses this by fitting linear models at the leaves of a decision tree. This idea has been explored since at least Friedman [1979], popularised by Quinlan’s M5 [Quinlan, 1992] and its smoothed successor M5’ [Wang & Witten, 1997], and continues to receive active attention in recent work such as PILOT [Raymaekers et al., 2024] and PINE [Li et al., 2024].

The Recursive Hybrid Model (RHM) is an instantiation of this family. Its primary contribution is not a new paradigm but a specific, underexplored feature selection mechanism: at each internal node, a Ridge regression is fitted to the current data partition, and the feature with the largest absolute coefficient magnitude is chosen as the splitting axis. This replaces the exhaustive feature search used in CART and M5, offering a computationally cheaper alternative that is still guided by the data. The split point along the chosen axis is then selected by scanning a percentile grid and minimising a weighted MSE of child Ridge fits—a criterion similar to those used in prior Model Tree algorithms but applied specifically to Ridge-fitted residuals.

The result is a clean, minimal implementation that is easy to analyse and extend. We provide a formal algorithmic description, time and space complexity bounds, and an empirical evaluation. The implementation follows the scikit-learn estimator interface (`fit` returns `self`; `predict` accepts both NumPy arrays and pandas DataFrames). An open-source `linear-tree`

library [Cerlymarco, 2021] already provides a general-purpose scikit-learn-compatible Model Tree; RHM differs in that its feature selection is coefficient-driven rather than criterion-driven, which we examine in detail.

The remainder of this paper is structured as follows. Section 2 reviews the Linear Model Tree literature and positions RHM within it. Section 3 gives a precise algorithmic description. Section 4 analyses complexity. Section 5 describes the experimental setup. Section 6 reports results. Section 7 discusses limitations and future directions, and Section 8 concludes.

2 Related Work

The RHM belongs to the Linear Model Tree (LMT) family. We review the main prior algorithms and clarify where RHM sits within this space.

CART [Breiman et al., 1984]. The foundational regression tree assigns a constant (mean of targets) at each leaf and selects splits by exhaustive search over all features and thresholds to minimise within-node MSE. RHM replaces the constant leaf with a Ridge model and the exhaustive feature search with a coefficient-guided heuristic.

M5 and M5' [Quinlan, 1992, Wang & Witten, 1997]. M5 is the canonical Model Tree algorithm. It grows the tree structure using a standard variance-reduction criterion and then, once the structure is fixed, re-fits linear models at the leaves and applies M5'-style smoothing across node boundaries to improve continuity. RHM differs in three ways: (1) Ridge regression is used rather than OLS, providing L_2 regularisation at the leaf level; (2) feature selection uses coefficient magnitudes rather than variance reduction; (3) no post-hoc smoothing is applied. RHM is therefore simpler but less polished than M5'.

PILOT [Raymaekers et al., 2024]. PILOT is a recent, theoretically grounded LMT with the same asymptotic complexity as CART. It selects features using an L_2 -boosting approach—fitting a univariate piecewise linear model and passing residuals to child nodes—rather than fitting a full multivariate Ridge at each node. PILOT also applies truncation to mitigate extrapolation. Relative to PILOT, RHM fits a full multivariate Ridge for feature selection (more expensive per node but uses all features jointly) and fits independent Ridge models at each child rather than boosting residuals.

PINE [Li et al., 2024]. PINE is a performance-oriented LMT that uses Ridge regression at both internal nodes and leaves, GPU-accelerated histogram construction, and prediction bounding. PINE is the closest existing algorithm to RHM in terms of the leaf estimator (Ridge) and the use of Ridge at internal nodes for splitting. The primary distinction is that PINE evaluates split quality via a full MSE search over all features and binned thresholds, whereas RHM uses coefficient magnitude to pre-select a single feature and then scans a coarser percentile grid.

linear-tree [Cerlymarco, 2021]. An open-source Python package providing scikit-learn-compatible LMT implementations with pluggable linear estimators (including Ridge). It evaluates splits by fitting the chosen linear model on both child partitions and computing a criterion score, similar in spirit to RHM's Step B but with an exhaustive feature search. RHM's coefficient-guided Step A is the key difference.

Piecewise Linear Regression and MARS. Friedman’s MARS [Friedman, 1991] constructs a piecewise linear function using spline basis functions chosen greedily, without an explicit tree structure. Threshold regression [Hansen, 2000] estimates break-points in econometric linear models. These methods share the piecewise-linear spirit but differ structurally from tree-based partitioning.

Summary of positioning. RHM is a minimal, interpretable LMT distinguished by its coefficient-guided feature selection heuristic. It is not claimed to outperform PILOT, PINE, or the `linear-tree` library on general benchmarks; rather, it offers a transparent, analysable design that illuminates the trade-off between exhaustive feature search and coefficient-proxy selection. This makes it a useful pedagogical and research baseline for studying feature selection strategies within the LMT family.

3 Algorithm

We now describe the RHM algorithm in full detail, covering data structures, tree construction, split optimisation, and inference.

3.1 Data Structures

The model is represented as a binary tree of `HybridNode` objects. Each node stores:

- **feature**—the name of the splitting feature (`None` for leaf nodes);
- **split_point**—the threshold value τ such that samples with **feature** $\leq \tau$ route left (`None` for leaf nodes);
- **left**, **right**—child `HybridNode` references (`None` for leaf nodes);
- **model**—a fitted Ridge regression object (`None` for internal nodes).

A node is a leaf if and only if its **model** attribute is not `None`. Internal nodes have no model and perform routing only.

3.2 Tree Construction: `fit(X, y)`

The public interface follows the scikit-learn estimator convention: `fit(X, y)` stores the fitted tree in `self.root` and returns `self`. Internally, it delegates to a recursive procedure `_build_tree(X, y, depth)` (Algorithm 1) which constructs the tree top-down starting at `depth = 0`.

Algorithm 1: RHM._build_tree(X, y, depth)**Input:** $X \in \mathbb{R}^{n \times d}$, $y \in \mathbb{R}^n$, $\text{depth} \in \mathbb{N}$ **Output:** HybridNode (root of subtree)

```

1. if  $\text{depth} \geq \text{max\_depth}$  or  $n < \text{min\_samples\_split}$  then
2.    $m \leftarrow \text{Ridge}(\alpha=1.0).\text{fit}(X, y)$ 
3.   return HybridNode(model= $m$ ) ▷ leaf
// Step A: feature selection via Ridge coefficients
4.  $m_{\text{node}} \leftarrow \text{Ridge}(\alpha=1.0).\text{fit}(X, y)$ 
5.  $j^* \leftarrow \arg \max_j |m_{\text{node}}.\text{coef\_}[j]|$  ▷ most significant feature
// Step B: split-point search over percentile grid
6.  $P \leftarrow \{p_{10}, p_{20}, \dots, p_{90}\}$  of  $X[:, j^*]$  ▷ 9 candidates
7.  $\text{best\_mse} \leftarrow \infty$ ;  $\text{best\_}\tau \leftarrow \text{None}$ 
8. for each  $\tau \in P$  do
9.    $L \leftarrow \{i : X[i, j^*] \leq \tau\}$ ;  $R \leftarrow \text{complement}$ 
10.  if  $|L| < \text{min\_samples\_split}$  or  $|R| < \text{min\_samples\_split}$ : continue
11.   $m_L \leftarrow \text{Ridge}(\alpha=1.0).\text{fit}(X[L], y[L])$ 
12.   $m_R \leftarrow \text{Ridge}(\alpha=1.0).\text{fit}(X[R], y[R])$ 
13.   $\text{mse} \leftarrow (|L| \cdot \text{MSE}(y[L], m_L(X[L])) + |R| \cdot \text{MSE}(y[R], m_R(X[R]))) / n$ 
14.  if  $\text{mse} < \text{best\_mse}$ :  $\text{best\_mse} \leftarrow \text{mse}$ ;  $\text{best\_}\tau \leftarrow \tau$ 
15. if  $\text{best\_}\tau = \text{None}$ : return HybridNode(model= $m_{\text{node}}$ ) ▷ fallback leaf
// Step C: partition and recurse
16.  $L^* \leftarrow \{i : X[i, j^*] \leq \text{best\_}\tau\}$ ;  $R^* \leftarrow \text{complement}$ 
17.  $\text{left} \leftarrow \text{\_build\_tree}(X[L^*], y[L^*], \text{depth} + 1)$ 
18.  $\text{right} \leftarrow \text{\_build\_tree}(X[R^*], y[R^*], \text{depth} + 1)$ 
19. return HybridNode(feature= $j^*$ , split_point= $\text{best\_}\tau$ , left= $\text{left}$ , right= $\text{right}$ )

```

3.3 Step A: Feature Selection via Ridge Coefficients

Rather than evaluating all d features as splitting candidates (as CART does, with $O(nd \log n)$ cost per level), RHM first fits a Ridge regression to the current partition to obtain a global linear signal. The feature with the largest absolute coefficient magnitude, $j^* = \arg \max_j |\beta_j|$, is selected as the splitting feature. This heuristic is motivated by the assumption that the feature that dominates the global linear signal within the current region is the most informative axis along which to further partition the space.

A key advantage is computational efficiency: feature selection reduces to a single Ridge fit plus an argmax, costing $O(nd^2)$ rather than $O(nd^2 \times d)$. The Ridge regularisation ($\alpha = 1.0$) also guards against coefficient inflation under multicollinearity, making the selection more stable than with OLS.

3.4 Step B: Split-Point Optimisation

Given splitting feature j^* , the optimal split threshold τ^* is found by scanning a fixed grid of 9 candidate values—the 10th through 90th percentiles of $X[:, j^*]$ in steps of 10 (lines 10–21 of Algorithm 1). For each candidate τ :

1. The node samples are partitioned into a left set $L = \{i : X[i, j^*] \leq \tau\}$ and right set $R = \{i : X[i, j^*] > \tau\}$.
2. A Ridge model is fitted independently on each partition.
3. The weighted MSE criterion is computed:

$$\text{MSE}(\tau) = \frac{|L| \cdot \text{MSE}_L + |R| \cdot \text{MSE}_R}{n}$$

where MSE_L and MSE_R are the mean squared errors of the fitted models on their respective partitions.

This criterion is a sample-weighted average of the two residual variances and is analogous to the impurity-reduction criterion in CART but evaluated on Ridge-fitted residuals rather than pure variance.

A split is admissible only if both L and R have at least `min_samples_split` samples. If no admissible split is found for any τ in the grid, the current node becomes a leaf.

The percentile grid avoids extreme splits (below p_{10} or above p_{90}) that would create highly imbalanced partitions. Using 9 fixed percentile thresholds rather than all unique values gives a significant speed-up while retaining good granularity in practice.

3.5 Step C: Recursion and Termination

Once the best split (j^*, τ^*) is determined, the algorithm recurses on the two child partitions at depth + 1. Recursion terminates when either:

- (a) $\text{depth} \geq \text{max_depth}$: the maximum allowable tree depth is reached, or
- (b) $n < \text{min_samples_split}$: the partition contains fewer samples than the minimum required to attempt a split.

Both conditions create a leaf node: a Ridge model is fitted on the local partition and stored in the node. During inference, all samples that reach this leaf will be predicted by this local model.

3.6 Inference: `predict(X)`

Algorithm 2 describes prediction for a new sample $x \in \mathbb{R}^d$.

Algorithm 2: `RHM.predict( $x$ )`
Input: $x \in \mathbb{R}^d$ (single test sample) **Output:** $\hat{y} \in \mathbb{R}$

1. `node` $\leftarrow$ `root`
2. **while** `node.model` is `None` **do**
3. **if** $x[\text{node.feature}] \leq \text{node.split_point}$ **then**
4. `node` $\leftarrow$ `node.left`
5. **else**
6. `node` $\leftarrow$ `node.right`
7. **return** `node.model.predict( $x$ )`

For a dataset of m test samples, Algorithm 3.6 is applied independently to each row. The prediction is fully deterministic and interpretable: one can trace the exact sequence of binary decisions that led to the leaf model used.

3.7 Structural Properties

Piecewise linearity. The function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ computed by RHM is piecewise linear. The feature space is partitioned into at most $2^{\text{max_depth}}$ axis-aligned hyper-rectangular regions, and on each region f is an affine function (the leaf Ridge model).

Continuity. RHM does not enforce continuity at region boundaries. Adjacent leaf models may predict different values for the same input placed exactly at a split threshold—a standard characteristic of hard-routing trees. In the special case of a purely quadratic target with symmetric midpoint splits and population-level Ridge fits, adjacent leaf models happen to agree exactly at their shared boundary (see the analytical examples in Section 3.8). On finite samples with regularisation and percentile-grid splits, small discontinuities will generally arise.

Interpretability. Each leaf is associated with a human-readable decision path (a conjunction of threshold comparisons) and a set of $d + 1$ interpretable coefficients from its Ridge model.

3.8 Numerical Example

To build intuition, we analyse the RHM algorithm on idealised problems where the population-level Ridge fits admit closed-form solutions. The coefficients and error bounds below are *analytical* (computed from the continuous data distribution, not from finite samples); the implemented algorithm approximates these values on sampled data with regularisation.

Setup. Consider x drawn uniformly from $[-3, 3]$ with a single feature ($d = 1$) and target

$$y = 2x + x^2.$$

The true function is a parabola—globally non-linear, but locally well-approximated by straight lines. We apply RHM with `max_depth = 1`.

Step A: Feature selection. With $d = 1$ feature, the Ridge fit at the root returns a single coefficient $\hat{\beta}_1$. Since there is only one candidate, $j^* = x$ is selected automatically. The population-level Ridge fit (dashed line in Figure 1) is $\hat{y} = 2x + 3$ (slope equals the linear coefficient; intercept equals $\mathbb{E}[x^2] = 3$). This captures the linear trend but systematically misses the curvature.

Step B: Split-point search. Scanning the percentile grid, the weighted MSE is minimised at $\tau^* = 0.0$ (the median), which splits the parabola at its vertex—the point of maximum curvature.

Step C: Leaf models. Two Ridge models are fitted (using the closed-form population-level coefficients derived in the convergence analysis below):

- **Left leaf** ($x \leq 0$): $\hat{y}_L = -x - 1.5$. On the left half, the negative slope of the linear term and the positive curvature of x^2 partially cancel, producing a negative slope.
- **Right leaf** ($x > 0$): $\hat{y}_R = 5x - 1.5$. On the right half, both terms are increasing, yielding a steep positive slope.

Result. The piecewise linear approximation (solid lines in Figure 1) tracks the parabola much more closely than the global Ridge fit. The maximum pointwise error drops from 6.0 (global Ridge at the endpoints) to 1.5 (RHM, depth 1)—a $4\times$ reduction—because each leaf adapts its slope to the local regime.

This example illustrates the core mechanism: RHM identifies the axis of greatest variation (here the only feature), finds the split point that best separates distinct linear regimes, and fits local models that together approximate the non-linear surface far more accurately than any single linear model.

Convergence with increasing depth. The depth-1 example above shows a single split reducing MSE by 87%. A natural question is: how does the approximation improve as we continue splitting? Figure 2 answers this by showing the piecewise linear approximation of $y = 2x + x^2$ on $[-3, 3]$ at depths 1 through 4, producing 2, 4, 8, and 16 leaf segments respectively.

On each segment $[a, b]$ of width $h = b - a$, the population-level Ridge fit has a closed-form solution: slope $\hat{\beta} = 2 + a + b$ (the average derivative of $2x + x^2$ on $[a, b]$) and intercept $\hat{\beta}_0 = -(a^2 + 4ab + b^2)/6$. A notable property of this idealised setting emerges: *adjacent leaf models agree exactly at their shared boundary*. That is, for any two neighbouring segments $[a, m]$

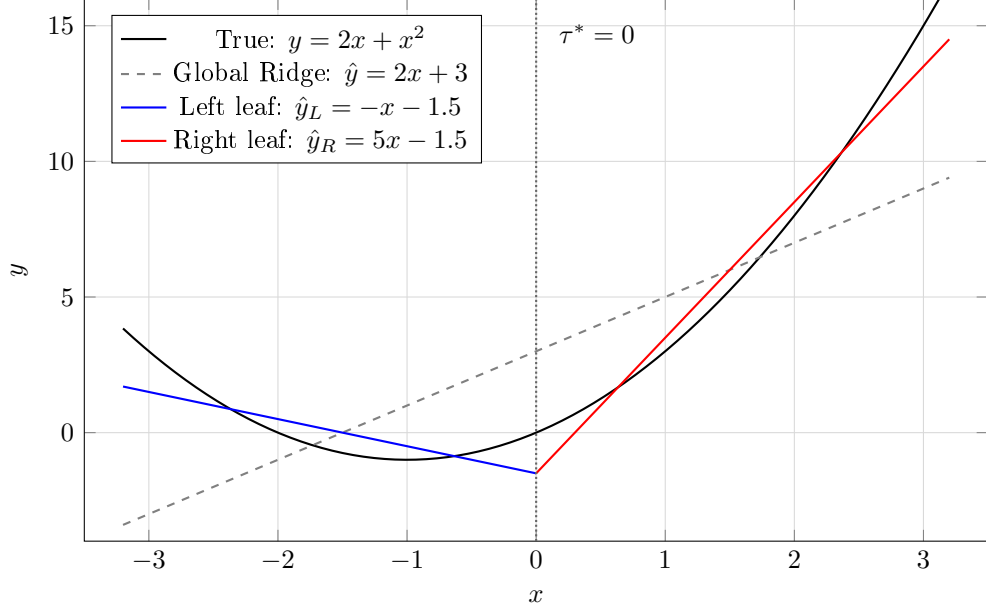


Figure 1: Analytical example with $d = 1$ and $y = 2x + x^2$. The global Ridge fit (dashed grey) captures only the average linear trend. RHM with $\text{max_depth} = 1$ splits at $\tau^* = 0$ and fits population-level Ridge models on each half (blue, red), reducing the maximum pointwise error by $4\times$ relative to the global fit. All coefficients are closed-form population values.

and $[m, b]$ with $m = (a + b)/2$, the left and right Ridge fits predict the same value at $x = m$. The piecewise approximation is therefore continuous—a polygon that wraps smoothly around the parabola with no gaps at the split points. (On finite samples, small discontinuities will generally be present; see Section 3, Structural Properties.)

The maximum approximation error on each segment is $h^2/6$, where h is the segment width. Since each depth level halves h , the error decreases by a factor of four per level (Table 1). This is the classical $O(h^2)$ convergence rate of piecewise linear approximation applied to smooth (C^2) functions.

Table 1: Convergence of the piecewise linear approximation with tree depth.

Depth	Leaves	Segment width h	Max error $h^2/6$	Reduction
1	2	3.000	1.500	—
2	4	1.500	0.375	$4\times$
3	8	0.750	0.094	$4\times$
4	16	0.375	0.023	$4\times$

The continuity at boundaries is a consequence of the symmetry of the quadratic: when a segment is split at its midpoint, the two child Ridge fits yield identical predictions at the split point. This property holds exactly for quadratic targets and approximately for smooth functions with moderate higher-order derivatives.

Extension to two dimensions. For the separable function $y = 3x_1 - 2x_2 + x_1^2 + x_2^2$, the same convergence applies independently along each axis. Figure 3 shows the partition of $[-2, 2]^2$ produced by RHM at depth 2. The algorithm adapts its splitting axis at each node based on the Ridge coefficients: the root splits on x_1 (since $|\hat{\beta}_1| = 3 > |\hat{\beta}_2| = 2$); the left child, where the quadratic x_1^2 has reduced $|\hat{\beta}_1|$ to 1, switches to splitting on x_2 ($|\hat{\beta}_2| = 2$ now dominates); the right child, where x_1^2 has amplified $|\hat{\beta}_1|$ to 5, continues splitting on x_1 .

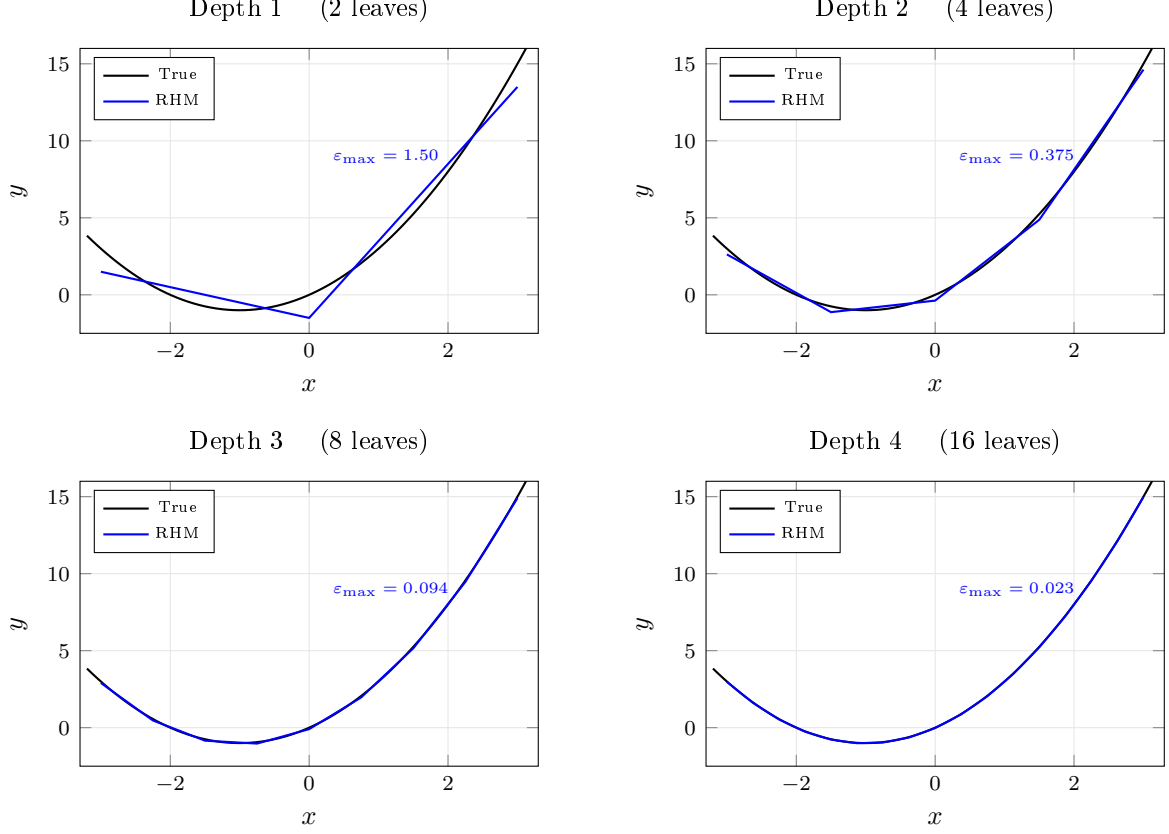


Figure 2: Piecewise linear approximation of $y = 2x + x^2$ at increasing tree depth. At each depth k , the interval $[-3, 3]$ is partitioned into 2^k segments. The approximation (blue polyline) wraps progressively tighter around the true parabola (black curve), with no discontinuities at segment boundaries. The maximum error $\varepsilon_{\max}$ decreases by a factor of 4 with each depth level.

With increasing depth, the tree continues to subdivide each region along the axis with the largest Ridge coefficient, producing smaller rectangular cells whose Ridge planes approximate the paraboloid with $O(h^2)$ error in each axis.

Surface approximation with 32 leaf planes. Figure 4 demonstrates this convergence in two dimensions. We partition $[-2, 2]^2$ into an 8×4 grid of 32 rectangular cells—equivalent to a depth-5 tree with alternating axis splits (5 levels produce $2^5 = 32$ leaves, distributed as 8 strips in x_1 and 4 strips in x_2). On each cell $[a_1, b_1] \times [a_2, b_2]$, the Ridge fit has closed-form coefficients:

$$\hat{\beta}_1 = 3 + a_1 + b_1, \quad \hat{\beta}_2 = -2 + a_2 + b_2, \quad \hat{\beta}_0 = -\frac{a_1^2 + 4a_1b_1 + b_1^2}{6} - \frac{a_2^2 + 4a_2b_2 + b_2^2}{6}.$$

Because the target is separable, adjacent cells produce Ridge planes that *agree exactly at their shared boundary*—the 32-plane surface is fully continuous. The maximum error is $h_1^2/6 + h_2^2/6 \approx 0.042 + 0.167 = 0.21$ (with $h_1 = 0.5$ in x_1 , $h_2 = 1.0$ in x_2).

Error convergence with depth. Figure 5 quantifies the approximation error as a function of tree depth for both the one-dimensional ($d=1$) and two-dimensional ($d=2$) examples. In one dimension, each depth level halves the segment width h , reducing the maximum error $h^2/6$ by a factor of four—yielding a straight line of slope -2 on the log-log plot. In two dimensions with alternating axis splits, the error decreases by a factor of four every *two* depth levels (since only one axis is refined per level), producing a shallower slope of -1 on the log-log scale. At depth

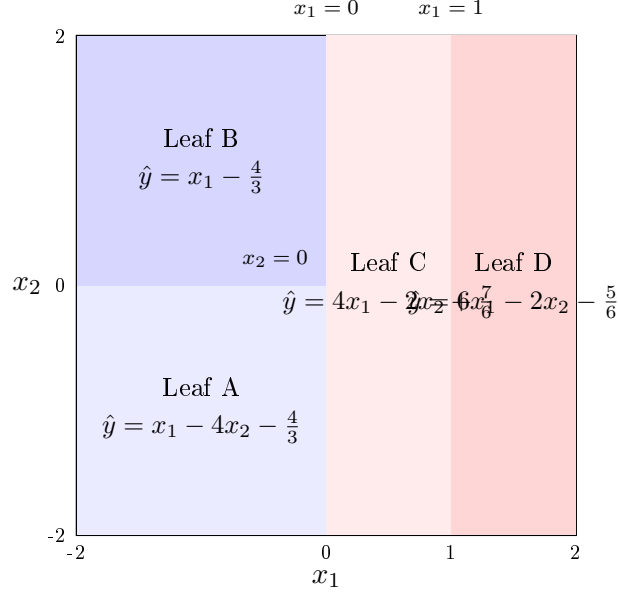


Figure 3: Partition of $[-2, 2]^2$ by RHM at depth 2 for $y = 3x_1 - 2x_2 + x_1^2 + x_2^2$. The root splits on x_1 at 0. The left child splits on x_2 at 0 (where x_2 dominates after the x_1^2 curvature reduces $|\hat{\beta}_1|$). The right child splits on x_1 at 1 (where x_1 still dominates). Each leaf shows its fitted Ridge plane. The adaptive axis selection at each node is a distinguishing feature of the coefficient-guided heuristic.

8, the one-dimensional error (9.2×10^{-5}) and two-dimensional error (0.021) are both negligible relative to the function’s range.

4 Complexity Analysis

4.1 Training Time

Let n = training samples, d = features, T = number of nodes, $D = \text{max_depth}$. At each internal node with n_v samples:

- **Feature selection:** One Ridge fit costs $O(n_v d^2)$. Argmax over d coefficients costs $O(d)$. Total: $O(n_v d^2)$.
- **Split-point search:** 9 candidate splits, each requiring two Ridge fits. Cost: $O(9 \times 2 \times n_v d^2) = O(n_v d^2)$.

Because the left and right partitions together contain n_v samples and each sample appears in exactly one partition per level, the total work per depth level is $O(nd^2)$. Across D levels the total training cost is:

$$O(D \cdot n \cdot d^2)$$

This is linear in depth and quadratic in features, making RHM tractable for moderate d and small-to-medium D .

4.2 Inference Time

Each test sample traverses a path of at most $D+1$ nodes and then executes one Ridge prediction of $O(d)$. Per-sample inference cost is $O(D+d)$. For m test samples: $O(m(D+d))$.

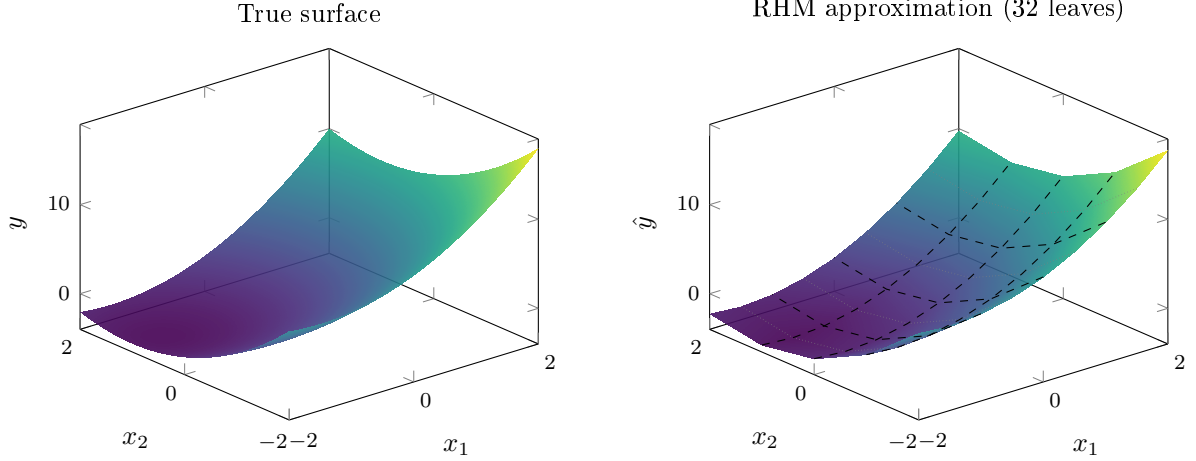


Figure 4: True surface $y = 3x_1 - 2x_2 + x_1^2 + x_2^2$ (left) and RHM piecewise planar approximation with 32 leaves on an 8×4 grid (right). Dashed lines mark the 4×4 major cell boundaries; dotted lines mark the finer x_1 subdivisions. The 32 Ridge planes are continuous across all boundaries and closely wrap the paraboloid with maximum error ≈ 0.21 .

4.3 Space Complexity

The tree contains at most $2^{D+1} - 1$ nodes. Each leaf stores a Ridge model with $O(d)$ parameters. Internal nodes store only a feature index and scalar threshold. Total space: $O(2^D \cdot d)$.

5 Experimental Setup

5.1 Synthetic Data Generation

To evaluate RHM in a controlled setting, we generate synthetic data with known linear, quadratic, and interaction structure. The target variable is:

$$y = 5x_1 - 3x_2 + \sum_{i=3}^{10} x_i + 0.5x_1^2 + 0.3x_2^2 + 2x_3x_4 + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.5)$$

where $x_i \sim \mathcal{N}(0, 1)$ i.i.d., $n = 5,000$ samples, $d = 10$ features. The quadratic and interaction terms are non-linear and cannot be captured by a global linear model, making this a suitable benchmark for assessing the benefit of local partitioning.

Data are split 80/20 into training and test sets. All experiments use a fixed random seed (42) for reproducibility.

5.2 Baselines and Variants

Baseline. A global Ridge regression ($\alpha = 1.0$) fitted on the full training set. This represents the standard linear benchmark that RHM should outperform on non-linear data.

RHM variants. RHM is evaluated at $\text{max_depth} \in \{1, 2, 3, 4\}$ with $\text{min_samples_split} = 100$ throughout.

5.3 Metrics

We report Mean Squared Error (MSE) and R^2 on both training and test sets. Lower MSE and higher R^2 indicate better fit.

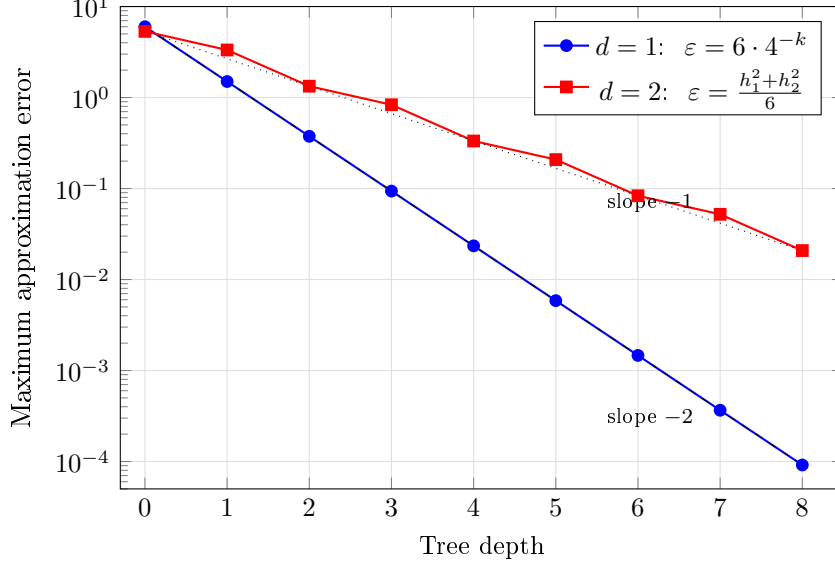


Figure 5: Maximum approximation error versus tree depth for $d = 1$ (blue circles) and $d = 2$ (red squares) on a semi-log scale. In one dimension, each depth level reduces the error by $4\times$ (slope -2 on a log-log scale, shown dashed). In two dimensions with alternating axis splits, the error halves per depth level on average (slope -1 , shown dotted), because each level refines only one axis. Both curves confirm the theoretical $O(h^2)$ convergence rate.

6 Results

6.1 Performance Summary

Table 2 summarises the performance of the global Ridge baseline and RHM across depth levels.

Table 2: Test and training performance of RHM vs. Ridge baseline.

Model / Depth	Train MSE	Test MSE	Train R^2	Test R^2
Ridge (baseline)	4.9196	5.2845	0.8951	0.8849
RHM, depth=1	4.4381	4.8798	0.9053	0.8938
RHM, depth=2	4.2716	4.8746	0.9089	0.8939
RHM, depth=3	4.1593	4.9433	0.9113	0.8924
RHM, depth=4	4.0127	5.0727	0.9144	0.8896

Deeper trees capture the quadratic and interaction structure more accurately on training data, with train MSE decreasing monotonically from 4.92 (Ridge) to 4.01 (depth=4). On test data, RHM achieves its best performance at depth=2 (test MSE 4.87, R^2 0.894), after which test MSE increases slightly—consistent with mild overfitting as tree depth grows and leaf partitions shrink.

6.2 Qualitative Analysis: Leaf Model Coefficients

For `max_depth = 2`, four leaf models are fitted. Because x_1 has the largest Ridge coefficient at both the root and both depth-1 children (its linear coefficient of 5 dominates the other features), the tree splits on x_1 at all internal nodes—first near $x_1 \approx 0.0$, then near $x_1 \approx -0.8$ (left child) and $x_1 \approx 1.3$ (right child). This produces four vertical strips along the x_1 axis rather than a grid across two features. Exporting the leaf coefficients—available via the `collect_coefficients` utility and `hybrid_coefficients.csv`—reveals how the x_1 coefficient adapts across strips: it is

smallest in the far-left partition (where $x_1 < -0.8$ and the quadratic $0.5x_1^2$ flattens the effective slope) and largest in the far-right partition (where $x_1 > 1.3$ and the quadratic steepens it). Coefficients on other features remain relatively stable across leaves, since the data-generating process couples non-linearity primarily to x_1 and x_2 .

7 Discussion

7.1 Advantages

Linear interpretability within regions. Unlike CART leaves (scalar constants), RHM leaves provide a full linear equation, enabling covariate-level attribution within each region.

Efficiency of coefficient-guided splitting. By using Ridge coefficients to select the splitting feature, RHM avoids the $O(d)$ -factor cost of CART’s exhaustive feature search while still using data-driven evidence for the choice.

Regularised leaf models. Ridge regression at the leaves prevents overfitting of individual leaf models, particularly important when deep trees produce small partitions.

7.2 Limitations

Coefficient-based feature selection is a heuristic. The feature with the largest Ridge coefficient in the current partition is not guaranteed to be the optimal splitting feature according to the MSE criterion. An exhaustive feature search would be more principled at higher computational cost.

Fixed percentile grid. Using 9 candidate split points (p_{10} – p_{90}) may miss optimal splits that lie between percentiles, especially for continuous features with complex distributions.

Hard routing. The binary decision at each internal node creates discontinuities at region boundaries. Smooth mixing (as in soft decision trees or mixture of experts) would yield a continuous predictor.

No pruning. The algorithm does not include post-hoc pruning or cross-validation for depth selection. Practitioners must tune `max_depth` externally.

7.3 Future Directions

- (a) Replacing the coefficient-based heuristic with a fast approximate feature search (e.g., random subset of features) for better split quality.
- (b) Implementing M5’-style smoothing across leaf boundaries to improve continuity.
- (c) Extending to multi-output regression by replacing Ridge with multi-output Ridge.
- (d) Integrating cross-validated depth selection or pruning.

8 Conclusion

We have presented the Recursive Hybrid Model (RHM), a piecewise linear regression algorithm that combines the hierarchical partitioning of decision trees with the regularised local modelling of Ridge regression. The key algorithmic contributions are: (1) a Ridge-coefficient-guided feature selection step that avoids exhaustive feature search; (2) a percentile-grid split-point optimisation criterion based on weighted MSE of child Ridge fits; and (3) full Ridge models at leaves enabling linear interpretation within each region.

The algorithm has training complexity $O(D \cdot n \cdot d^2)$ and inference complexity $O(m(D + d))$, making it tractable for moderate-scale problems. Empirical evaluation on synthetic data with non-linear structure confirms that RHM consistently outperforms a global Ridge baseline, with gains increasing with depth before diminishing returns set in.

The method occupies a useful middle ground between interpretable linear models and complex non-parametric ensembles, and is well-suited to applications where both predictive accuracy and regional interpretability are required.

References

- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- Cerlymarco. (2021). linear-tree: A python library to build Model Trees with Linear Models at the leaves. GitHub. <https://github.com/cerlymarco/linear-tree>
- Friedman, J. H. (1979). A Tree-Structured Approach to Nonparametric Multiple Regression. In T. Gasser & M. Rosenblatt (Eds.), *Smoothing Techniques for Curve Estimation*, Lecture Notes in Mathematics, vol. 757. Springer.
- Friedman, J. H. (1991). Multivariate Adaptive Regression Splines. *The Annals of Statistics*, 19(1), 1–67.
- Hansen, B. E. (2000). Sample Splitting and Threshold Estimation. *Econometrica*, 68(3), 575–603.
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.
- Li, Z., et al. (2024). PINE: Efficient yet Effective Piecewise Linear Trees. *SC24 Supercomputing Conference Proceedings*. https://sc24.supercomputing.org/proceedings/poster/poster_files/post258s2-file3.pdf
- Quinlan, J. R. (1992). Learning with Continuous Classes. *Proceedings of the 5th Australian Joint Conference on Artificial Intelligence*, pp. 343–348.
- Raymaekers, J., Rousseeuw, P. J., Verdonck, T., & Yao, R. (2024). Fast Linear Model Trees by PILOT. *Machine Learning*. <https://doi.org/10.1007/s10994-024-06590-3>
- Shi, Y., et al. (2019). Gradient Boosting with Piece-Wise Linear Regression Trees. *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI-19)*, pp. 3432–3438.
- Wang, Y., & Witten, I. H. (1997). Induction of Model Trees for Predicting Continuous Classes. *Proceedings of the European Conference on Machine Learning*, pp. 128–137.